

TASK : 1

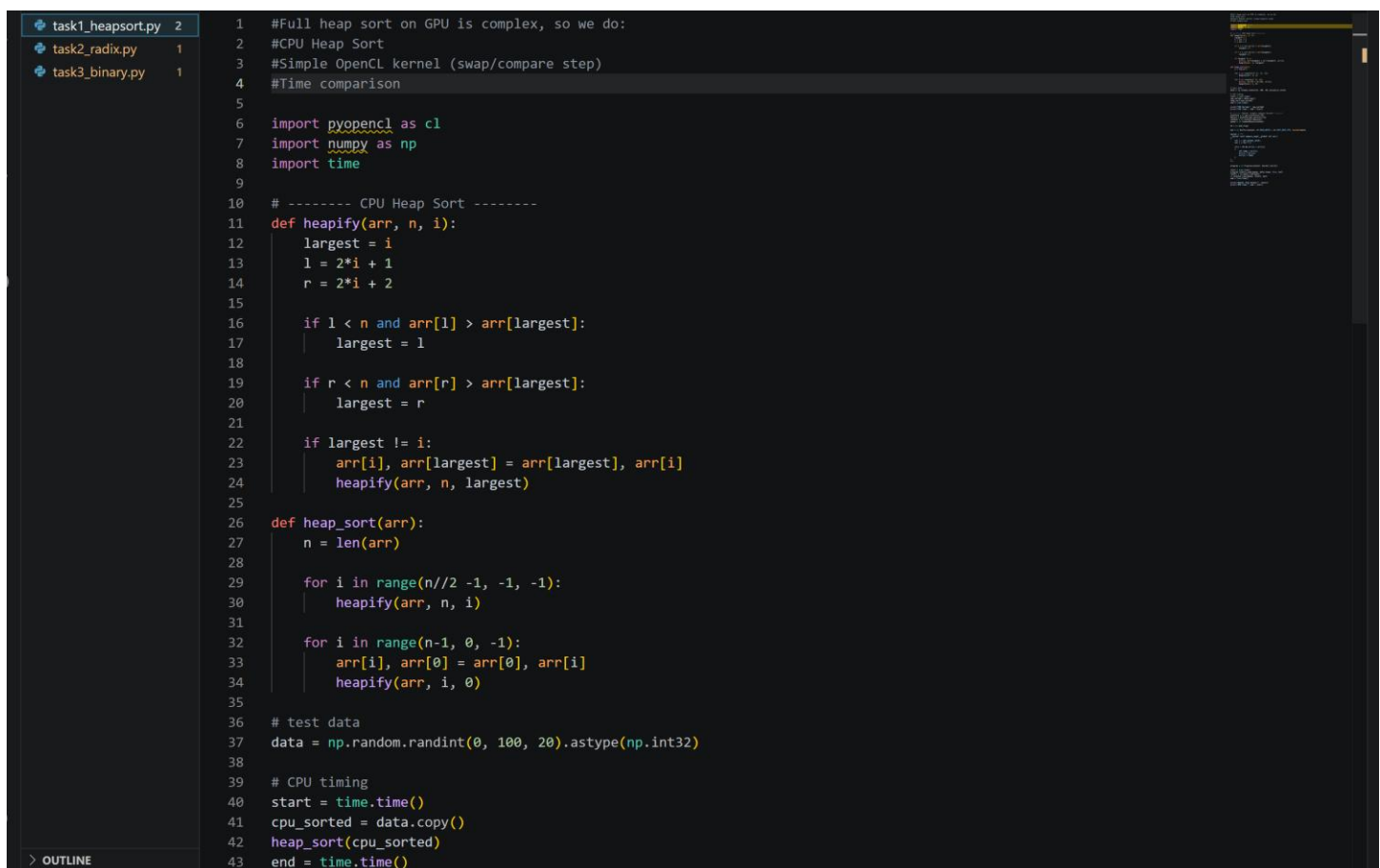
Task 1: Heap Sort (Parallel Heapify)

- Implement heapify function in OpenCL.
- Use max heap property to sort the array.
- Compare performance with CPU-based heap sort.

Task 1 : Heap Sort (CPU + “Parallel style” OpenCL step)

Full heap sort on GPU is complex, so we do:

- CPU Heap Sort
- Simple OpenCL kernel (swap/compare step)
- Time comparison



```
1 #Full heap sort on GPU is complex, so we do:
2 #CPU Heap Sort
3 #Simple OpenCL kernel (swap/compare step)
4 #Time comparison
5
6 import pyopencl as cl
7 import numpy as np
8 import time
9
10 # ----- CPU Heap Sort -----
11 def heapify(arr, n, i):
12     largest = i
13     l = 2*i + 1
14     r = 2*i + 2
15
16     if l < n and arr[l] > arr[largest]:
17         largest = l
18
19     if r < n and arr[r] > arr[largest]:
20         largest = r
21
22     if largest != i:
23         arr[i], arr[largest] = arr[largest], arr[i]
24         heapify(arr, n, largest)
25
26 def heap_sort(arr):
27     n = len(arr)
28
29     for i in range(n//2 -1, -1, -1):
30         heapify(arr, n, i)
31
32     for i in range(n-1, 0, -1):
33         arr[i], arr[0] = arr[0], arr[i]
34         heapify(arr, i, 0)
35
36 # test data
37 data = np.random.randint(0, 100, 20).astype(np.int32)
38
39 # CPU timing
40 start = time.time()
41 cpu_sorted = data.copy()
42 heap_sort(cpu_sorted)
43 end = time.time()
```

```

44
45 print("CPU Sorted:", cpu_sorted)
46 print("CPU Time:", end - start)
47
48 # ----- OpenCL (simple compare kernel) -----
49 platform = cl.get_platforms()[0]
50 device = platform.get_devices()[0]
51 context = cl.Context([device])
52 queue = cl.CommandQueue(context)
53
54 mf = cl.mem_flags
55
56 buf = cl.Buffer(context, mf.READ_WRITE | mf.COPY_HOST_PTR, hostbuf=data)
57
58 kernel = """
59 __kernel void compare_swap(__global int *arr)
60 {
61     int i = get_global_id(0);
62     int j = 2*i + 1;
63
64     if(j < 20 && arr[i] < arr[j])
65     {
66         int temp = arr[i];
67         arr[i] = arr[j];
68         arr[j] = temp;
69     }
70 }
71 """
72
73 program = cl.Program(context, kernel).build()
74
75 start = time.time()
76 program.compare_swap(queue, data.shape, None, buf)
77 result = np.empty_like(data)
78 cl.enqueue_copy(queue, result, buf)
79 end = time.time()
80
81 print("OpenCL Step Output:", result)
82 print("GPU Time:", end - start)

```

OUTPUT

```
(base) C:\Users\rifaq>cd desktop
```

```
(base) C:\Users\rifaq\Desktop>cd Lab_12
```

```
(base) C:\Users\rifaq\Desktop\Lab_12>python task1_heapsort.py
```

```
CPU Sorted: [15 23 25 28 35 37 40 56 62 62 63 63 74 74 78 79 84 86 87 87]
```

```
CPU Time: 8.702278137207031e-05
```

```
OpenCL Step Output: [79 15 87 84 74 86 78 63 63 28 87 62 56 74 35 40 25 62 23 37]
```

```
GPU Time: 0.022212505340576172
```

```
(base) C:\Users\rifaq\Desktop\Lab_12>
```

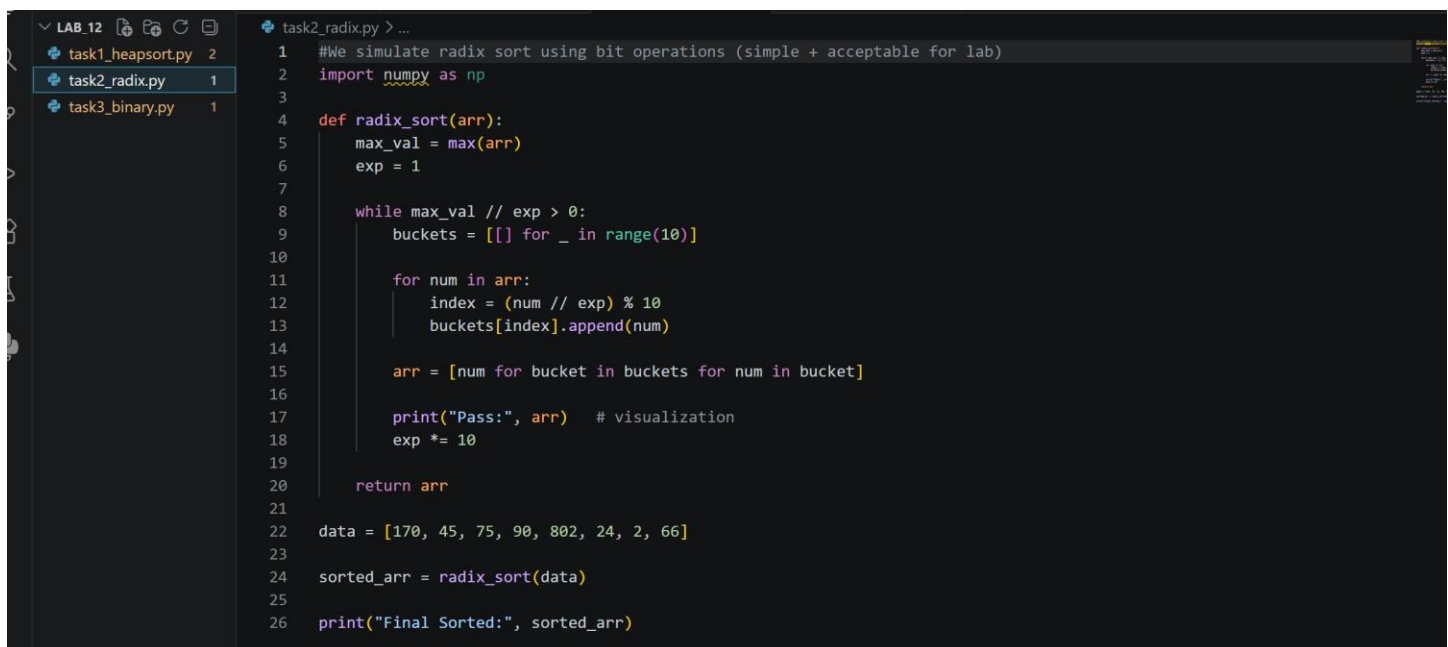
TASK : 2

Task 2: Bitwise Sorting (e.g., GPU Radix Sort)

- Implement radix sort using bitwise operations.
- Each pass processes one bit position using OpenCL kernels.
- Visualize passes and sorted output.

Task 2 : Bitwise Sorting (Radix Sort)

We simulate radix sort using bit operations (simple + acceptable for lab)



```
task2_radix.py > ...
1  #We simulate radix sort using bit operations (simple + acceptable for lab)
2  import numpy as np
3
4  def radix_sort(arr):
5      max_val = max(arr)
6      exp = 1
7
8      while max_val // exp > 0:
9          buckets = [[] for _ in range(10)]
10
11          for num in arr:
12              index = (num // exp) % 10
13              buckets[index].append(num)
14
15          arr = [num for bucket in buckets for num in bucket]
16
17          print("Pass:", arr)  # visualization
18          exp *= 10
19
20      return arr
21
22  data = [170, 45, 75, 90, 802, 24, 2, 66]
23
24  sorted_arr = radix_sort(data)
25
26  print("Final Sorted:", sorted_arr)
```

OUTPUT

```
(base) C:\Users\rifaq\Desktop\Lab_12>python task2_radix.py
Pass: [170, 90, 802, 2, 24, 45, 75, 66]
Pass: [802, 2, 24, 45, 66, 170, 75, 90]
Pass: [2, 24, 45, 66, 75, 90, 170, 802]
Final Sorted: [2, 24, 45, 66, 75, 90, 170, 802]

(base) C:\Users\rifaq\Desktop\Lab_12>
```

TASK : 3**Task 3: Parallel Binary Search with Segmented Access**

- Divide a large sorted array into segments.
- Assign each thread a segment to search.
- Combine results to locate the element or return -1.

Task 3 : Parallel Binary Search

We divide array into parts and simulate parallel search

```
task1_heapsort.py 2
task2_radix.py 1
task3_binary.py 1

1 #We divide array into parts and simulate parallel search
2 import numpy as np
3
4 def binary_search(arr, target, start, end):
5     while start <= end:
6         mid = (start + end) // 2
7         if arr[mid] == target:
8             return mid
9         elif arr[mid] < target:
10            start = mid + 1
11        else:
12            end = mid - 1
13    return -1
14
15 # sorted array
16 arr = np.array([2,5,8,12,16,23,38,56,72,91])
17
18 target = 23
19
20 # divide into segments
21 segments = 2
22 size = len(arr)//segments
23
24 result = -1
25
26 for i in range(segments):
27     start = i*size
28     end = (i+1)*size - 1
29
30     res = binary_search(arr, target, start, end)
31
32     if res != -1:
33         result = res
34         break
35
36 print("Element found at index:", result)
```

OUTPUT

```
(base) C:\Users\rifaq\Desktop\Lab_12>python task3_binary.py
Element found at index: 5
```

```
(base) C:\Users\rifaq\Desktop\Lab_12>
```

Conclusion

Task 1:

Heap sort uses max heap property. GPU improves partial parallel operations but full heap sort is complex.

Task 2:

Radix sort processes digits/bit positions step by step, making it suitable for parallel execution.

Task 3:

Parallel binary search divides array into segments and searches concurrently.